



UNIVERSIDADE FEDERAL DE SANTA CATARINA
CAMPUS TRINDADE
DEPARTAMENTO DE INFORMÁTICA E ESTATÍSTICA
CURSO CIÊNCIAS DA COMPUTAÇÃO

Lívia Corazza Ferrão - 21202119
Bianca Mazzuco Verzola - 22202621

INE5429 - Segurança da computação

**Needham-Schroeder e Denning-Sacco:
Implementação, Análise de Vulnerabilidades e Ataques**

Florianópolis
2026

SUMÁRIO

1. Introdução.....	3
2. Desenvolvimento teórico.....	3
2.1. Conceitos fundamentais.....	3
2.1.1. Centro de Distribuição de Chaves (KDC).....	3
2.1.2. Nonces e Marcas Temporais.....	4
2.2. Protocolo Needham-Schroeder (1978).....	4
2.2.1 Passos.....	4
2.2.2. Exemplo Numérico.....	5
2.2.3 Propriedades de Segurança.....	6
2.3 Vulnerabilidade do NS: Ataque de Repetição.....	6
2.3.1 Cenário do Ataque.....	6
2.4. Protocolo Denning-Sacco.....	7
2.4.1 Passos.....	7
2.4.2. Exemplo Numérico.....	8
2.4.3 Limitações do DS e Discussão.....	8
3. Experimento.....	8
3.1 Objetivos da Implementação.....	8
3.2 Arquitetura.....	9
3.3 Comunicação e formato das mensagens.....	9
3.4 Pseudocódigo.....	10
3.4.1 Protocolo Needham-Schroeder.....	10
3.4.2 Ataque de Repetição contra o NS (Mallory).....	11
3.4.3 Protocolo Denning-Sacco.....	11
3.4.4 Ataque de Repetição contra o DS (Mallory).....	11
3.5 Execução e resultados observados.....	12
4. Declaração de uso de Inteligência Artificial.....	13
5. Referências.....	14

1. Introdução

A distribuição segura de chaves criptográficas é um dos desafios fundamentais da criptografia moderna. Em sistemas distribuídos, onde múltiplas entidades precisam se comunicar de forma confidencial e autenticada, surge a necessidade de estabelecer chaves de sessão de maneira segura, sem que os segredos de longo prazo sejam expostos em cada troca de mensagens.

Este trabalho trata dos protocolos de distribuição de chaves de sessão baseados em um servidor de autenticação confiável, com foco em dois esquemas amplamente estudados na literatura: o protocolo Needham-Schroeder (NS), proposto em 1978 [1], e sua versão aprimorada, o protocolo Denning-Sacco (DS), proposto em 1981 [2]. Ambos os protocolos utilizam criptografia simétrica e um terceiro confiável, denominado Centro de Distribuição de Chaves (KDC, do inglês Key Distribution Center), para garantir a distribuição autenticada de chaves de sessão entre usuários.

A motivação para a escolha deste tema reside em três aspectos principais:

- Relevância histórica e teórica: o Needham-Schroeder é um dos protocolos criptográficos mais estudados e citados da história, e sua vulnerabilidade ao ataque de repetição (“replay attack”), descoberta apenas anos após a publicação, ilustra com clareza como a análise formal de protocolos é essencial, mesmo para esquemas aparentemente simples.
- Oportunidade de aprendizado prático: implementar os protocolos e reproduzir o ataque de repetição oferece uma experiência concreta de como falhas sutis de design podem comprometer completamente a segurança de um sistema.
- Conexão com sistemas reais: conceitos derivados destes protocolos são encontrados em sistemas amplamente utilizados, como o Kerberos [3], que resolve algumas das mesmas limitações aqui exploradas.

Ao longo deste relatório, são apresentados os fundamentos teóricos dos dois protocolos, a vulnerabilidade inerente ao NS original e sua exploração prática, além do planejamento da implementação experimental.

2. Desenvolvimento teórico

2.1. Conceitos fundamentais

Antes de descrever os protocolos, apresentamos as noções básicas necessárias para sua compreensão.

2.1.1. Centro de Distribuição de Chaves (KDC)

Um KDC é uma entidade confiável que funciona como intermediário de confiança no sistema. Quando um usuário se cadastra, ele recebe uma chave secreta exclusiva com o KDC, chamada de chave de longo prazo, porque permanece a mesma por muito tempo, ao contrário das chaves de sessão, que mudam a cada conversa. Assim, o KDC guarda uma chave diferente para cada usuário: uma para A (chamada K_A), outra para B (chamada K_B), e assim por diante. Quando A e B desejam se comunicar, mas ainda não possuem uma chave em comum, eles recorrem ao KDC. Ele gera uma chave de sessão temporária K_s e a entrega para cada um separadamente: para A, cifrando K_s com K_A (que só A conhece); para B, cifrando K_s com K_B (que só B conhece). Dessa forma, ninguém além do destinatário consegue ler a chave de sessão [4].

Essa abordagem centralizada simplifica o gerenciamento de chaves. Sem um KDC, cada par de usuários precisaria manter uma chave exclusiva entre si, e o número de pares cresce de forma quadrática com o número de usuários n , o que se escreve como $O(n^2)$. Com o KDC, cada usuário mantém apenas uma chave pessoal com o KDC, totalizando $O(n)$ chaves no sistema. O KDC se encarrega de gerar chaves de sessão temporárias conforme necessário [4].

2.1.2. Nonces e Marcas Temporais

Um problema central em protocolos de autenticação é garantir que uma mensagem é realmente nova e não uma cópia de uma mensagem legítima do passado que um atacante gravou e está reproduzindo [4]. Dois mecanismos são usados para isso:

- Nonce (Number used ONCE): valor aleatório gerado por uma das partes e inserido em uma mensagem. Se a resposta incluir o nonce corretamente transformado, demonstra que o remetente processou a mensagem atual e não apenas repetiu uma mensagem antiga. O protocolo NS utiliza nonces [1].
- Marca temporal (Timestamp): indica o momento exato de criação de uma mensagem. O receptor pode rejeitar mensagens cujo timestamp seja muito antigo. O protocolo DS adiciona timestamps ao esquema NS para corrigir sua vulnerabilidade [2].

2.2. Protocolo Needham-Schroeder (1978)

O protocolo Needham-Schroeder de chave simétrica [1] foi um dos primeiros protocolos formalmente descritos para distribuição autenticada de chaves de sessão. Seu objetivo é permitir que A e B estabeleçam uma chave de sessão K_s de forma segura, com duas garantias principais: confidencialidade, ou seja, somente A e B terão acesso a K_s ; e autenticidade, ou seja, cada parte tem a certeza de que está se comunicando com quem afirma ser. Tudo isso é válido desde que o KDC seja confiável. Nas seções a seguir, os usuários A e B correspondem, respectivamente, a Alice e Bob.

2.2.1 Passos

O protocolo NS envolve cinco passos [1][4]. Ao longo desta seção, utiliza-se a notação $\{M\}_K$ para indicar que a mensagem M foi cifrada com a chave K.

Passo 1: Solicitação de uma chave de sessão

$A \rightarrow KDC: A, B, N_A$

Alice envia ao KDC sua própria identidade (**A**), a identidade de Bob (**B**) e um nonce (**N_A**), indicando que deseja estabelecer uma sessão segura com Bob. O nonce é um valor aleatório gerado exclusivamente para esta solicitação, permitindo que Alice verifique posteriormente que a resposta do KDC corresponde a este pedido específico e não a uma mensagem anterior reutilizada.

Passo 2: Resposta do KDC

$KDC \rightarrow A: \{N_A, B, K_s, \{K_s, A\}_{K_B}\}_{K_A}$

O KDC gera uma nova chave de sessão (**K_s**) e responde a Alice com uma mensagem criptografada utilizando a chave secreta compartilhada entre Alice e o KDC (**K_A**).

Essa mensagem contém:

- o nonce N_A , permitindo que Alice confirme que a resposta corresponde à sua solicitação;
- a identidade de Bob (B);
- a chave de sessão (K_s);
- um ticket destinado a Bob, contendo K_s e a identidade de Alice, criptografado com K_B .

Como o ticket está protegido por K_B , apenas Bob poderá decifrá-lo.

Passo 3: Envio do ticket para Bob

$$A \rightarrow B : \{K_s, A\}_{K_B}$$

Após verificar o nonce recebido, Alice encaminha o ticket para Bob. Como o ticket está criptografado com K_B , Alice não consegue alterá-lo nem visualizar seu conteúdo.

Quando Bob recebe o ticket, utiliza sua chave secreta K_B para obter a chave de sessão K_s e confirmar que a sessão foi criada para comunicação com Alice.

Passo 4: Desafio de autenticação

$$B \rightarrow A : \{N_B\}_{K_s}$$

Bob decifra o ticket com K_B , obtendo K_s e a identidade de Alice. Para confirmar que Alice realmente conhece a chave de sessão recém-estabelecida, Bob gera um novo nonce (N_B) e o envia criptografado com K_s .

Como apenas quem possui K_s consegue decifrar essa mensagem, esse passo funciona como um desafio de autenticação.

Passo 5: Resposta ao desafio

$$A \rightarrow B : \{N_B - 1\}_{K_s}$$

Alice decifra a mensagem recebida, obtém o nonce N_B e responde enviando $N_B - 1$, criptografado com a mesma chave de sessão K_s .

Ao verificar que o valor recebido corresponde corretamente ao desafio enviado, Bob conclui que Alice conhece K_s , autenticando sua identidade e estabelecendo uma sessão segura.

2.2.2. Exemplo Numérico

A seguir, ilustramos o protocolo com valores concretos, simplificados para fins didáticos. Os parâmetros iniciais são: $K_A = 0xAB12$ (chave de Alice com o KDC), $K_B = 0xCD34$ (chave de Bob com o KDC), $N_A = 42$ (nonce gerado por Alice) e $N_B = 99$ (nonce gerado por Bob). A chave de sessão $K_s = 0xEF56$ é gerada pelo KDC.

1. Alice envia ao KDC: (Alice, Bob, 42). O KDC recebe o pedido e identifica as partes e o nonce.
2. O KDC responde para Alice:
 $\{42, \text{Bob}, 0xEF56, \{0xEF56, \text{Alice}\}_{0xCD34}\}_{0xAB12}$

- Alice decifra com K_A e verifica que $N_A == 42$.
- Alice repassa o ticket para Bob:
 $\{0xEF56, Alice\}_{0xCD34}$
 Bob decifra com K_B e obtém $K_s = 0xEF56$ e a identidade de Alice.
 - Bob envia o desafio para Alice:
 $\{99\}_{0xEF56}$
 Alice decifra com K_s e obtém $N_B = 99$.
 - Alice responde com $\{98\}_{0xEF56}$ ($N_B - 1 = 99 - 1 = 98$)
 Bob verifica: $99 - 1 == 98$, confirmando a autenticação de Alice.

2.2.3 Propriedades de Segurança

O protocolo NS, quando executado corretamente com chaves válidas e sem interferência externa, oferece as seguintes garantias de segurança [1][4]:

- Confidencialidade da chave de sessão: K_s é sempre transmitida cifrada, nunca em claro. Para Alice, está protegida por K_A ; para Bob, está protegida por K_B dentro do ticket.
- Autenticação mútua: o mecanismo de desafio-resposta dos passos 4 e 5 garante que ambas as partes possuem K_s . Bob só aceita a sessão após Alice responder corretamente ao desafio N_B .
- Autorização pelo KDC: Bob só aceita uma sessão com Alice se o ticket vier cifrado com K_B . Como apenas o KDC e Bob conhecem K_B , nenhum outro usuário pode fabricar um ticket válido.

2.3 Vulnerabilidade do NS: Ataque de Repetição

Em 1981, Denning e Sacco identificaram uma falha crítica no protocolo NS [2]: o protocolo não dispõe de mecanismo para garantir que a chave de sessão K_s seja recente. O ticket $\{K_s, A\}_{K_B}$ repassado por Alice a Bob no passo 3 não contém nenhuma informação sobre o momento em que foi gerado, e Bob não tem como verificar sua validade temporal. Essa ausência viabiliza o chamado ataque de repetição (“replay attack”): um intruso que tenha gravado mensagens de sessões anteriores pode reenviá-las posteriormente e comprometer a segurança de Bob.

2.3.1 Cenário do Ataque

Suponha que um intruso tenha monitorado a rede e gravado o tráfego de uma sessão legítima anterior entre A e B, incluindo a mensagem do passo 3: $\{K_s, A\}_{K_B}$. Mesmo que essa sessão tenha terminado há horas ou dias, o intruso pode reutilizar essa mensagem para realizar um ataque de repetição (replay attack) contra B.

O intruso pode então proceder da seguinte forma:

- Reenvia para Bob o ticket gravado, como se fosse Alice iniciando uma nova sessão:
 Intruso $\rightarrow$ B : $\{K_{s_antiga}, A\}_{K_B}$ (mensagem gravada anteriormente)
- Bob decifra o ticket com K_B , obtém K_{s_antiga} e emite um desafio:

$B \rightarrow A : \{ N_{B'} \}_{K_{s_antiga}}$

3. Se o intruso conhece K_{s_antiga} , por exemplo, porque a chave foi comprometida anteriormente ou vazou após o término da sessão, ele poderá responder ao desafio de B:

$I \rightarrow B : \{ N_{B'} - 1 \}_{K_{s_antiga}}$

Resultado: o intruso consegue se passar por A, levando B a acreditar que está estabelecendo uma sessão legítima com Alice.

O problema central é que Bob não tem como verificar que K_s é antiga: o ticket é criptograficamente válido (decifra com K_B sem erros) e não contém qualquer informação temporal. Bob o aceita como legítimo e inicia a sessão sem saber que está se comunicando com um intruso, e não com Alice [2][5].

2.4. Protocolo Denning-Sacco

Para corrigir a vulnerabilidade do NS, Denning e Sacco propuseram a inclusão de uma marca temporal T , gerada pelo KDC, dentro do ticket [2]. Ao receber o ticket, Bob não verifica apenas a validade criptográfica, mas também se T está dentro de uma janela de tempo aceitável. Tickets cujo timestamp exceda esse limite são automaticamente rejeitados, eliminando diretamente a possibilidade de ataque de repetição.

2.4.1 Passos

Diferentemente do protocolo NS, o protocolo DS utiliza apenas três passos [2][4].

Passo 1: Solicitação de uma chave de sessão

$A \rightarrow KDC : A, B$

Alice solicita ao KDC uma chave de sessão com Bob. Ao contrário do protocolo NS, não há nonce nesta etapa, pois a validade da chave será garantida pelo timestamp gerado pelo KDC.

Passo 2: Resposta do KDC

$KDC \rightarrow A : \{ B, K_s, T, \{ K_s, A, T \}_{K_B} \}_{K_A}$

O KDC gera K_s e registra o horário atual como timestamp T . A resposta para Alice é cifrada com K_A , e o ticket para Bob é cifrado com K_B , ambos contendo T . A diferença essencial em relação ao NS está na inclusão de T no ticket: $\{ K_s, A, T \}_{K_B}$ em vez de $\{ K_s, A \}_{K_B}$.

Passo 3: Envio do ticket para Bob

$A \rightarrow B : \{ K_s, A, T \}_{K_B}$

Alice repassa o ticket para Bob. Bob decifra com K_B , extrai K_s , a identidade de Alice e o timestamp T , e verifica se a diferença entre T e o horário local atual está dentro do limite Δt configurado. Caso o ticket esteja expirado, a sessão é rejeitada; caso contrário, é aceita. Vale notar que, ao contrário do NS, o DS não inclui handshake explícito de autenticação mútua: a

autenticação é implícita, pois se Bob aceita o ticket (timestamp válido e deciframento bem-sucedido), sabe que o KDC gerou K_s recentemente para Alice.

2.4.2. Exemplo Numérico

A seguir, ilustramos o protocolo com valores concretos, simplificados para fins didáticos. Os parâmetros iniciais são: $K_A = 0xAB12$ (chave de Alice com o KDC), $K_B = 0xCD34$ (chave de Bob com o KDC) e $K_s = 0xEF56$ (chave de sessão gerada pelo KDC). O timestamp gerado pelo KDC é $T = 2026-06-25\ 14:30:00$, e a janela de aceitação é $\Delta t = 5$ minutos.

1. Alice envia ao KDC: (Alice, Bob). Não há nonce nesta solicitação.
2. O KDC responde para Alice:
 $\{Bob, 0xEF56, T, \{0xEF56, Alice, 2026-06-25\ 14:30:00\}_{0xCD34}\}_{0xAB12}$
Alice decifra com K_A e obtém K_s e o ticket para Bob.
3. Alice repassa o ticket para Bob:
 $\{0xEF56, Alice, 2026-06-25\ 14:30:00\}_{0xCD34}$
Bob decifra com K_B , obtém $K_s = 0xEF56$, a identidade de Alice e $T = 14:30:00$.
Bob verifica: $|14:30:05 - 14:30:00| = 5s < 5\ min$. Sessão aceita.

Para ilustrar a defesa contra o ataque de repetição, suponha que o intruso tente reutilizar o mesmo ticket horas depois: Bob verificaria $|17:00:00 - 14:30:00| = 2,5h > 5\ min$ e rejeitaria o ticket como expirado.

2.4.3 Limitações do DS e Discussão

Embora o DS resolva o problema de repetição, ele introduz uma nova dependência: os relógios de Alice, Bob e do KDC devem estar sincronizados. Em caso de dessincronização significativa, por exemplo, causada por um ataque de sincronização de tempo, o mecanismo pode falhar: tickets legítimos podem ser rejeitados por um receptor cujo relógio esteja adiantado, ou tickets expirados podem ser aceitos por um receptor com relógio atrasado. Trata-se de uma troca característica no projeto de protocolos criptográficos, em que a correção de uma vulnerabilidade pode introduzir novas dependências [4].

O Kerberos v5 [3] pode ser visto como a evolução natural dessas ideias: ele combina timestamps (para garantir a validade temporal das chaves, como no DS) com handshakes de autenticação mútua (para confirmar que ambas as partes possuem a chave, como no NS), e adiciona mecanismos de renovação de tickets (TGT) para evitar que o usuário precise se reautenticar a cada nova sessão.

3. Experimento

3.1 Objetivos da Implementação

A implementação teve como objetivos:

- Simular os dois protocolos (NS e DS) com KDC, Alice e Bob executando como servidores/clientes TCP reais, cada um em uma thread independente.

- Demonstrar o ataque de repetição contra o NS, com Mallory reproduzindo o ticket e a chave de sessão capturados de uma execução legítima anterior.
- Demonstrar a defesa do DS contra o mesmo tipo de ataque, aguardando a expiração real do timestamp antes do replay.
- Registrar e exibir no console todas as mensagens trocadas entre as partes, em cada um dos três cenários.

3.2 Arquitetura

O código-fonte completo está disponível em:

<https://github.com/Livia-ferrao/INE5429-Seguranca/tree/main/TrabalhoNsDs>

O sistema é composto por nove módulos Python, divididos entre lógica pura de protocolo e papéis de rede (que usam sockets). Essa separação permite ler e entender os cinco/três passos de cada protocolo isoladamente da comunicação TCP:

- `crypto_utils.py`: chaves de longo prazo (K_A , K_B) pré-distribuídas; geração de K_s ; cifra/decifra AES-CBC (chave de 128 bits) com padding PKCS7 e IV aleatório, via a biblioteca `cryptography`; codificação base64 para embutir bytes cifrados em JSON.
- `net_utils.py`: framing de mensagens JSON sobre socket TCP: cada mensagem é prefixada por 4 bytes (big-endian) com o tamanho do payload, já que TCP é um fluxo de bytes sem delimitação nativa de mensagens.
- `ns_protocol.py`: lógica pura dos 5 passos do Needham-Schroeder: construção/interpretação de cada mensagem.
- `ds_protocol.py`: lógica pura dos 3 passos do Denning-Sacco, incluindo a geração e verificação do timestamp T contra a janela ΔT .
- `kdc.py`: servidor TCP (porta 9000) que recebe pedidos NS1/DS1, delega a construção da resposta para `ns_protocol.py/ds_protocol.py` e devolve a mensagem cifrada correspondente.
- `bob.py`: dois servidores TCP separados, porta 9001 para o handshake NS e porta 9002 para o DS, para que os dois protocolos possam ser demonstrados lado a lado sem se misturar.
- `alice.py`: inicia os protocolos NS e DS, conectando-se ao KDC e depois a Bob via sockets TCP reais.
- `mallory.py`: papel do intruso: reconecta diretamente ao(s) socket(s) de Bob (sem passar pelo KDC ou por Alice) e reenvia bytes reais de um ticket capturado em uma execução legítima anterior.
- `run_demo.py`: orquestrador: sobe KDC e Bob em threads, executa os três cenários em sequência (NS legítimo, replay contra o NS, DS legítimo seguido de replay) e imprime um resumo final.

3.3 Comunicação e formato das mensagens

A comunicação implementada é feita por sockets TCP reais em 127.0.0.1, com as seguintes portas fixas:

- KDC: porta 9000, trata os pedidos NS1 (pedido NS) e DS1 (pedido DS).

- Bob, canal NS: porta 9001, recebe NS3 (ticket), envia NS4 (desafio) e recebe NS5 (resposta ao desafio).
- Bob, canal DS: porta 9002, recebe DS3 (ticket com timestamp) e verifica se o timestamp está dentro da janela DELTA_T.

Cada mensagem trafega como um objeto JSON, com os trechos cifrados (payload do KDC, ticket para Bob, desafio/resposta) embutidos em base64, precedido por um cabeçalho de 4 bytes indicando o tamanho do payload (net_utils.send_json / recv_json). A cifra usada em todos os trechos protegidos é AES-128 em modo CBC com padding PKCS7 e um vetor de inicialização (IV) aleatório por mensagem, implementada em crypto_utils.py com a biblioteca cryptography.

3.4 Pseudocódigo

3.4.1 Protocolo Needham-Schroeder

// KDC

função kdc_responder(A, B, N_A):

```

    K_s ← gerar_chave_aleatória()
    ticket_B ← encrypt({K_s, A}, K_B)
    resposta ← encrypt({N_A, B, K_s, ticket_B}, K_A)
    retorna resposta

```

// Alice

função alice_ns():

```

    N_A ← gerar_nonce()
    enviar_para_KDC(A, B, N_A) // Passo 1
    resp ← receber_do_KDC()
    {N_A', B', K_s, ticket_B} ← decrypt(resp, K_A)
    verificar: N_A' == N_A e B' == B
    enviar_para_B(ticket_B) // Passo 3
    desafio ← receber_de_B()
    N_B ← decrypt(desafio, K_s)
    enviar_para_B(encrypt(N_B - 1, K_s)) // Passo 5

```

// Bob

função bob_ns():

```

    ticket ← receber_de_A() // Passo 3
    {K_s, A'} ← decrypt(ticket, K_B)
    N_B ← gerar_nonce()
    enviar_para_A(encrypt(N_B, K_s)) // Passo 4
    resposta ← receber_de_A()
    N_B' ← decrypt(resposta, K_s)
    verificar: N_B' == N_B - 1 // Passo 5
    // Sessão estabelecida com A usando K_s

```

3.4.2 Ataque de Repetição contra o NS (Mallory)

// Mallory grava o passo 3 de uma sessão anterior

função mallory_replay_ns(ticket_antigo, K_s_antiga):

 // Fase 1: repetição do ticket

 enviar_para_B(ticket_antigo) // Passo 3'

 desafio ← receber_de_B() // Passo 4': B gera N_B'

 N_B' ← decrypt(desafio, K_s_antiga) // Mallory conhece K_s_antiga

 enviar_para_B(encrypt(N_B' - 1, K_s_antiga)) // Passo 5'

 // Bob aceita a sessão: vulnerabilidade confirmada!

3.4.3 Protocolo Denning-Sacco

// KDC

função kdc_responder_ds(A, B):

 K_s ← gerar_chave_aleatória()

 T ← timestamp_atual()

 ticket_B ← encrypt({K_s, A, T}, K_B)

 resposta ← encrypt({B, K_s, T, ticket_B}, K_A)

 retorna resposta

// Alice

função alice_ds():

 enviar_para_KDC(A, B) // Passo 1

 resp ← receber_do_KDC()

 {B', K_s, T, ticket_B} ← decrypt(resp, K_A) // Passo 2

 verificar: B' == B

 enviar_para_B(ticket_B) // Passo 3

// Bob

função bob_ds():

 ticket ← receber_de_A() // Passo 3

 {K_s, A', T} ← decrypt(ticket, K_B)

 T_agora ← timestamp_atual()

 se |T_agora - T| > DELTA_T:

 rejeitar("Ticket expirado")

 retorna

 // Sessão aceita apenas se ticket for recente

3.4.4 Ataque de Repetição contra o DS (Mallory)

// Mallory grava o ticket do passo 3 de uma sessão DS anterior

função mallory_replay_ds(ticket_capturado):

 enviar_para_B(ticket_capturado) // Passo 3'

Diferente do ataque contra o NS, aqui não há passos 4' e 5': o DS não possui desafio-resposta, então Mallory apenas reenvia o ticket capturado e aguarda a decisão de Bob. A função `bob_ds()` decifra o ticket normalmente, pois ele continua criptograficamente válido, mas calcula a diferença entre o horário atual e o timestamp `T` embutido no ticket ($\text{diff} = |\text{timestamp_atual}() - T|$). Como esse valor excede a janela `DELTA_T`, Bob rejeita o ticket e a sessão não é criada.

3.5 Execução e resultados observados

A execução (`python3 run_demo.py`) sobe o KDC e os dois servidores de Bob em threads e roda três cenários em sequência, aguardando a abertura efetiva de cada porta TCP antes de prosseguir:

- Cenário 1: NS legítimo: Alice e Bob completam os cinco passos do Needham-Schroeder e estabelecem uma sessão; o console confirma "sessão estabelecida" do lado de Bob.

- Cenário 2: Replay contra o NS: Mallory reutiliza o ticket e a `K_s` capturados no Cenário 1 (premissa: a chave foi comprometida após o fim da sessão legítima). Como o ticket não carrega nenhuma informação temporal, Bob o aceita normalmente.

- Cenário 3: DS com defesa: Alice e Bob completam o handshake do Denning-Sacco. O script então aguarda de verdade a expiração da janela `DELTA_T` (3 segundos na configuração de demonstração) antes de Mallory reenviar exatamente o mesmo ticket capturado. Desta vez Bob rejeita o ticket com o motivo "ticket expirado", confirmando a defesa.

O resumo impresso ao final da execução consolida o resultado esperado a partir da teoria da Seção 2: o NS, por não validar a idade do ticket, aceita o replay; o DS, por embutir e validar um timestamp no próprio ticket, rejeita o mesmo tipo de ataque assim que a janela de validade se esgota.

A Figura 1 mostra a saída real do terminal para essa execução, com as mensagens trocadas nos três cenários e o resumo final impresso pelo script.

```

[Bob] escutando em 127.0.0.1:9002
[Bob] escutando em 127.0.0.1:9001
[KDC] escutando em 127.0.0.1:9000

=====
CENÁRIO 1: Needham-Schroeder -- handshake legítimo entre Alice e Bob
=====
[Alice-NS] Passo 1 -> KDC: A=alice, B=bob, N_A=10393048378738477568
[Alice-NS] Passo 2 <- KDC: resposta {N_A,B,K_s,ticket_B}_K_A recebida
[Alice-NS] Passo 3 -> Bob: repassando ticket {K_s,A}_K_B
[Alice-NS] Passo 4 <- Bob: desafio N_B=10181849656548514642
[Alice-NS] Passo 5 -> Bob: resposta N_B-1=10181849656548514641
[Bob-NS] sessão estabelecida com 'alice' (K_s=2456189c...)

=====
CENÁRIO 2: Ataque de repetição (replay attack) contra o NS
=====
Premissa: Mallory gravou o ticket do cenário 1 e, posteriormente,
comprometeu a K_s daquela sessão (ex.: vazamento após o término).
[Mallory] Passo 3' -> Bob: reenviando ticket capturado {K_s_antiga,A}_K_B
[Mallory] Passo 4' <- Bob: desafio N_B=1211686039261532706
[Mallory] Passo 5' -> Bob: resposta N_B-1=1211686039261532705 (usando K_s_antiga vazada)
[Mallory] replay NS concluído -- verifique acima se Bob-NS estabeleceu a sessão
[Bob-NS] sessão estabelecida com 'alice' (K_s=2456189c...)

=====
CENÁRIO 3: Denning-Sacco -- handshake legítimo, depois replay
=====
[Alice-DS] Passo 1 -> KDC: A=alice, B=bob
[Alice-DS] Passo 2 <- KDC: resposta {B,K_s,T,ticket_B}_K_A recebida
[Alice-DS] Passo 3 -> Bob: repassando ticket {K_s,A,T}_K_B (T=1783272110.077)
[Bob-DS] sessão estabelecida com 'alice' (K_s=91683010...)

Aguardando 4s para o ticket expirar (janela DELTA_T=3s) antes do replay do Mallory...
Mallory reenvia exatamente o mesmo ticket DS capturado no início do cenário 3:
[Mallory] Passo 3' -> Bob: reenviando ticket DS capturado {K_s,A,T}_K_B
[Mallory] replay DS concluído -- verifique acima se Bob-DS aceitou ou rejeitou o ticket
[Bob-DS] ticket de 'alice' REJEITADO -- ticket expirado: |agora - T| = 4.2s excede a janela de 3s

=====
RESUMO
=====
NS (sem timestamp): replay aceito por Bob      -> vulnerabilidade confirmada
DS (com timestamp): replay REJEITADO -> defesa confirmada (ver Bob-DS acima)

```

Figura 1 – Saída do terminal para python3 run_demo.py, mostrando os três cenários (NS legítimo, replay contra o NS, DS com defesa) e o resumo final.

4. Declaração de uso de Inteligência Artificial

Durante o desenvolvimento deste trabalho, foram utilizadas as ferramentas de Inteligência Artificial ChatGPT e Claude como apoio ao processo de aprendizagem e desenvolvimento. Essas ferramentas auxiliaram na compreensão de conceitos, esclarecimento de dúvidas, revisão e sugestões de código, além de apoiar a revisão textual da documentação. Todo o conteúdo produzido com auxílio dessas ferramentas foi revisado, validado e adaptado pelos autores.

5. Referências

- [1] R. M. Needham e M. D. Schroeder. "Using Encryption for Authentication in Large Networks of Computers". *Communications of the ACM*, 21(12):993–999, dezembro de 1978.
- [2] D. E. Denning e G. M. Sacco. "Timestamps in Key Distribution Protocols". *Communications of the ACM*, 24(8):533–536, agosto de 1981.
- [3] B. C. Neuman e T. Ts'o. "Kerberos: An Authentication Service for Computer Networks". *IEEE Communications Magazine*, 32(9):33–38, setembro de 1994.
- [4] D. R. Stinson e M. B. Paterson. *Cryptography: Theory and Practice*, 4a edição. CRC Press, 2018.
- [5] G. Lowe. "Breaking and Fixing the Needham-Schroeder Public-Key Protocol Using FDR". In: *Tools and Algorithms for the Construction and Analysis of Systems, TACAS 1996. Lecture Notes in Computer Science*, vol. 1055, pp. 147–166. Springer, 1996.